

Feature-Augmented Graph Neural Networks for Out-of-Distribution Detection of Collusive Market Manipulation: A Multi-Cohort Study

Sumit Sontakke

Department of Computer Science, PES University, Bengaluru, India

pes2pge24ds111@stu.pes.edu

June 5, 2026

Abstract

Surveillance systems that flag collusive trading must do more than fit the data they were trained on; they must hold up when the parameters, seeds, and even the data-generating process shift. We report a multi-stage study of a two-layer GraphSAGE manipulation detector augmented with six taxonomy-grounded per-trader features, evaluated across three increasingly difficult generalisation regimes. On a fifty-run out-of-distribution cohort with parameter-disjoint manipulation scenarios, the feature-augmented model lifts edge-level ROC AUC from 0.638 (two topology features) to 0.984 and trader recall from 43% to 83% at 74% purity. The same features, fed instead into a gradient-boosted second-stage classifier on top of the baseline graph network, reach pooled trader AUC 0.968 under leave-one-run-out cross-validation and clear the 99%-precision bar at 62% recall. On a fifteen-run cross-generator cohort whose manipulators are produced by a different agent-based simulator entirely, the augmented network retains edge AUC 0.842 while the two-feature baseline collapses to chance (0.518), demonstrating that the engineered features carry manipulation-essential signal across simulation regimes. A leave-one-family-out test, however, reveals an unexpected contrast: the second-stage classifier maintains AUC between 0.91 and 0.99 on the held-out family, while the end-to-end network collapses to chance in every fold. The simpler bolt-on stack therefore generalises across manipulation styles better than the more expressive end-to-end model. We discuss the implications for surveillance system design and outline the open questions for cross-distribution operating-point calibration.

Keywords: Market surveillance, collusive manipulation, graph neural networks, out-of-distribution generalisation, agent-based simulation, feature ablation, leave-one-family-out validation.

1 Introduction

Detecting collusive manipulation in equity markets is a problem whose character has changed substantially over the past decade. Standalone abusive behaviour by a single trader can often be caught with simple rules on order volume and price impact, but coordinated activity by a ring of accounts, a clique acting through proxy fronts, or a group deliberately splitting trades across counterparties leaves only a relational signature. Each individual trader may look unremarkable; the conspiracy is visible only when one looks at *who trades with whom, when, and in what coordinated pattern*. This makes the detection task naturally graph-shaped and motivates the use of graph neural networks (GNNs), which can encode both per-trader features and the structure of inter-trader interactions in a unified representation.

Yet expressive models are not, by themselves, enough. A surveillance system is judged not on its training distribution but on its operating distribution, and the operating distribution is almost always different. Real markets shift by season, by ticker, by regulator-induced microstructure changes; even when the underlying generator is held fixed, the parameter envelope of any specific period is unlikely to match the model’s training period. Worse, the synthetic cohorts on which academic detectors are typically validated come from a single simulator, and a model that succeeds within that simulator’s parameter range may still fail when exposed to a structurally different generator. Robust deployment therefore requires multiple, increasingly demanding generalisation tests rather than a single in-distribution accuracy number.

This paper presents a study that does exactly that. We build a two-layer GraphSAGE model on a calibrated synthetic market substrate and study its behaviour under three nested out-of-distribution regimes: a parameter-disjoint test cohort drawn from the same generator, a cross-generator cohort whose manipulators come from a different agent-based simu-

lator (ABIDES [11]), and a leave-one-family-out evaluation in which one of the three manipulation families (clique, ring, mixed) is held out from training entirely. To contextualise the model-only result, we also evaluate a second-stage gradient-boosted classifier that consumes the GraphSAGE output together with six per-trader engineered features designed to capture manipulation-essential structure.

Our contributions are fourfold. First, we present a six-feature engineered representation of trader-level manipulation signatures and show that several of its components carry standalone discriminative power comparable to or exceeding that of the baseline graph network. Second, we show that simply concatenating these six features onto the GNN node vector and re-training from scratch lifts the model’s within-generator OOD edge AUC from 0.638 to 0.984. Third, we report a cross-generator evaluation on ABIDES manipulator agents in which the augmented model holds AUC 0.842 (versus chance for the unaugmented baseline), confirming that the engineered features carry signal across simulators. Fourth, and most surprisingly, we report a leave-one-family-out evaluation in which the simple second-stage classifier remains accurate (AUC 0.91–0.99) while the end-to-end augmented network collapses to chance. We discuss why this appears to be the case and what it implies for surveillance system design.

The remainder of this paper is organised as follows. Section 2 surveys related work in manipulation detection and graph learning. Section 3 formalises the problem. Section 4 describes the synthetic cohorts used. Section 5 describes the four detector architectures we compare. Section 7 reports the within-generator, cross-generator, and family-disjoint results. Section 8 discusses what the comparison implies. Sections 9 and 12 cover limitations and conclude.

2 Related Work

Market manipulation detection has a long literature, much of it rule- or threshold-driven. Early work focused on layering, spoofing, and pump-and-dump patterns visible in order-book microstructure [1]. Supervised classifiers later appeared, treating manipulation detection as a per-order or per-trader classification problem with hand-engineered statistics; Cao and co-authors, for instance, applied particle-swarm-tuned learners to labelled real and synthetic exchange data [2]. The shift from per-order to relational reasoning has been driven by the observation that genuinely collusive activity tends to evade trader-marginal classifiers: any single account looks legitimate. **Graph-based fraud and anomaly detection** has emerged as a natural response. Recent work

has applied graph neural networks to financial-fraud datasets and demonstrated meaningful lifts over non-relational baselines [3], though most benchmarks remain within-distribution.

Graph neural networks for inductive learning. We adopt GraphSAGE [4] as the backbone because it supports inductive node-representation learning, which matters in the surveillance setting where deployment graphs differ from training graphs in both the identity of nodes and the size of the graph. Other attention-based architectures such as GAT [5] and spectral methods such as GCN [6] are widely studied alternatives, but their inductive properties on dynamic, partially observed market graphs are less well characterised.

Imbalanced classification pervades surveillance: only a small fraction of any cohort is manipulative. We use focal loss [7] for the GNN’s edge-prediction head, which down-weights easy negatives and concentrates the gradient on hard positives. For the second-stage classifier we use gradient-boosted trees [8], which handle modest training set sizes and heterogeneous features well without extensive tuning.

Out-of-distribution evaluation. The need to evaluate classifiers on data drawn from a distinct distribution has been emphasised in image classification [9] and made systematic in domain-generalisation benchmarks such as WILDS [10]. The natural analogue in the surveillance setting is to vary not only the seeds and parameters but also the underlying data-generating process. To that end, we use ABIDES [11], a discrete-event agent-based market simulator, to produce a held-out cohort whose generator differs from our primary synthesiser.

Calibrated synthetic markets. The baseline synthesiser used in this work follows the stylised facts of asset returns set out by Cont [12], including heavy-tailed return distributions, volatility clustering, and the absence of linear autocorrelation in returns. Calibration to a real exchange (the Indian NSE bhavcopy time series) was performed in an earlier project phase to ensure the synthetic market is at least first-moment-faithful to real conditions.

3 Problem Formulation

We consider a daily slice of an equity market populated by N traders, each of whom emits orders that may or may not match against the limit order book. Trades resulting from matched orders form a directed weighted graph $G = (V, E)$ in which each trader is a node and an edge from a to b records that a sold to b some quantity over the slice. A subset $M \subset V$ of traders is deemed manipulative; in the synthetic regime considered here this labelling is exact, with

each M belonging to one of three manipulation *families*:

- **Clique**: a small set of accounts that coordinate to push or absorb price, typically routed through one or two shared front counterparties.
- **Ring**: a larger set of accounts that pass positions around a closed cycle, creating heavy fictitious volume.
- **Mixed**: a scenario combining elements of both, in which some accounts participate in clique behaviour and others in ring behaviour during the same slice.

The detector’s task is to output a score $\hat{y}: V \rightarrow [0, 1]$ ranking each trader by manipulator probability, ideally such that the score’s ordering is stable under seed, parameter and generator shifts. We evaluate via three metrics: edge-level ROC AUC (informative because the underlying classifier emits edge probabilities first), per-family trader recall at a locked threshold, and per-family trader purity (precision among flagged traders) at the same threshold.

4 Data and Cohort Design

4.1 Training Cohort

The training cohort consists of five days of synthetic market sessions; each day comprises fifty independent runs sampled from a parameter envelope deliberately wider than the typical evaluation suite. Each run contains approximately 500 traders, of which 0.05%–2% are manipulators belonging to a randomly drawn family. Manipulation duration is sampled between 10 and 60 minutes within the trading session. The five-day cohort design supports a continual warm-start training protocol in which the model retains weights from one day’s training to seed the next, with smaller per-day epoch budgets after the initial day to limit catastrophic forgetting.

4.2 Within-Generator OOD Cohort

The primary out-of-distribution evaluation cohort consists of fifty runs drawn from a parameter envelope strictly outside the training distribution: session duration 60–90 minutes, manipulation ratio 2–5%, core manipulator count between 16 and 25. Random seeds are held out from the training distribution. Approximately one-third of the runs are clique scenarios, one-third ring, and one-third mixed.

Table 1: Cohort summary.

Cohort	Runs	Traders/run	Manip./run
Training (5-day)	250	~500	~10–20
Within-generator OOD	50	~500	~16–25
ABIDES pilot	3	~125	6
ABIDES expanded	15	~125	6

4.3 Cross-Generator Cohort

To stress the model further, we generated an additional eighteen runs using ABIDES, an agent-based discrete-event market simulator distinct from the primary synthesiser. Three pilot runs (one per family) verified that the data schema matched, and fifteen further runs (five seeds across three families, all at a held-out calibration date) form the statistical-power expansion. Each ABIDES run contains approximately 125 active traders and six manipulator agents per run. The ABIDES manipulator agents—`CollusiveCliqueAgent`, `RingTraderAgent`, and `FrontAccountAgent`—are implementation-level approximations of the same three taxonomies used in the primary synthesiser, but the underlying microstructure (limit order book dynamics, message ordering, exchange agent behaviour) differs substantially.

4.4 Dataset Statistics

Table 1 summarises the three cohorts used in this study. The training cohort is the largest by run count; the within-generator OOD cohort is comparable in per-run size; the cross-generator ABIDES cohort is smaller per run because the ABIDES agent population is configured around 125 active traders rather than the 500 used in the primary synthesiser, but it remains large enough to expose distribution shift without becoming trivial.

5 Detector Architectures

5.1 Baseline GraphSAGE (v1)

Our baseline detector is a two-layer GraphSAGE network operating on the per-run directed trade graph. Each layer applies a `SAGEConv` aggregator (we use the `max`-pool variant) with hidden dimensions (256, 128, 64) and a dropout rate of 0.3. An MLP head consumes the concatenated source-destination embeddings and edge-level features—lead-lag correlation, signed imbalance, traded volume, interaction count, pre- and post-window drift, and VWAP imbalance—and emits a per-edge manipulation probability. The model is trained with focal loss ($\alpha=0.85, \gamma=2.0$) and a custom early-stopping signal based on the per-family

trader recall on a held-out validation set rather than the more conventional validation cross-entropy. This training protocol was chosen for its alignment with the surveillance operating metric.

Each graph’s node feature vector consists of just two features: the trader’s total executed quantity and the trader’s number of distinct counterparties. The edge feature vector contains the eight quantities listed above, computed over a one-minute bucket.

After training, edge probabilities are projected to trader scores via the rule $\hat{y}(v) = 0.7 \max_e p(e) + 0.3 \text{mean}_{e \in \text{top}_3} p(e)$, which gave the best per-family balance in earlier ablations.

5.2 Engineered Manipulation-Signature Features

We define six per-trader features each scoped to that trader’s busiest five-minute order window. Below, O_v denotes the set of orders placed by trader v during the session, $B_v \subseteq O_v$ denotes their burst window, and T_v denotes the trades to which v was a party during B_v .

1. **Burst concentration** $\phi_1(v) = |B_v|/|O_v|$.
2. **Side entropy in burst** $\phi_2(v) = -p \log_2 p - (1-p) \log_2 (1-p)$ where p is the buy fraction within B_v .
3. **Counterparty HHI in burst** $\phi_3(v) = \sum_k s_k^2$, where s_k is the share of T_v attributable to counterparty k .
4. **Order quantity coefficient of variation** $\phi_4(v) = \sigma(q)/\mu(q)$ where q is the vector of order quantities of v .
5. **Top-partner trade share** $\phi_5(v) = \max_k s_k$, using the same shares as ϕ_3 .
6. **Co-active top-quantile count** $\phi_6(v)$: the number of other top-activity-quantile traders whose burst window overlaps that of v .

The standalone discriminative power of each feature on the primary OOD cohort is given in Table 2. The strongest single feature is ϕ_3 (counterparty HHI) at AUC 0.800; the weakest is ϕ_2 (side entropy) at 0.550. Notably, three of the six exceed the baseline GraphSAGE’s pooled trader AUC of 0.793, indicating that the features alone carry signal comparable to the end-to-end network.

5.3 Tier-2 Stacked Classifier

Our first augmented detector is a stacked classifier. We take the baseline GraphSAGE’s per-trader score as input x_0 , concatenate the six engineered

Table 2: Standalone ROC AUC of engineered features on the primary OOD cohort.

Feature	AUC
Counterparty HHI in burst (ϕ_3)	0.800
Top-partner trade share (ϕ_5)	0.775
Co-active top-quantile count (ϕ_6)	0.744
Burst concentration (ϕ_1)	0.704
Order qty coefficient of variation (ϕ_4)	0.666
Side entropy in burst (ϕ_2)	0.550
Baseline GraphSAGE v1 trader score (reference)	0.793

features ($\phi_1, \dots, \phi_6$) to form a seven-element input vector, and train a gradient-boosted classifier ($n = 120, \text{depth}=3, \eta=0.08, \text{subsample}=0.85$) on this representation under leave-one-run-out cross-validation. The classifier emits a trader-level manipulator probability that combines the graph network’s view (x_0) with the trader-marginal features.

5.4 Feature-Augmented GraphSAGE (v4)

The second augmented detector folds the engineered features into the graph network’s node representation instead of using them as a downstream classifier. The node feature vector grows from two dimensions to eight: [volume, counterparties, $\phi_1, \dots, \phi_6$]. The remaining architecture, training schedule, and loss are held fixed, so the only variable between the baseline network and v4 is the node feature set.

This means v4 cannot warm-start from the baseline checkpoint—the first **SAGEConv** layer’s weight matrix changes shape. v4 therefore trains from scratch on the same five-day cohort.

5.5 Trader-Level Projection Rule

All four detectors emit per-trader scores. For the two graph networks, which natively produce per-edge probabilities, we project to traders via a weighted blend of each trader’s maximum incident-edge probability and the mean of their top three. Formally, for trader v with incident-edge probabilities $\{p_e\}_{e \in E(v)}$:

$$\hat{y}(v) = 0.7 \max_{e \in E(v)} p_e + 0.3 \frac{1}{3} \sum_{e \in \text{top}_3(v)} p_e. \quad (1)$$

The 0.7/0.3 split was selected in an earlier project phase by maximising per-family trader recall on a held-out validation cohort. A pure-max projection overweights single noisy edges and hurts purity; a pure-mean projection dilutes the strong signal from clear manipulators. Equation (1) balances these tendencies.

Algorithm: Continual warm-start GraphSAGE training.

1. **Input:** cohort days $D_0, \dots, D_4$; node-feature spec Φ
2. **Output:** per-day checkpoints $\theta_0, \dots, \theta_4$
3. Initialise model $\theta \leftarrow \text{random}$
4. **for** $d = 0, \dots, 4$ **do**
5. $G_d \leftarrow \{\text{build_graph}(r, \Phi) \mid r \in D_d\}$
6. $(G_d^{\text{tr}}, G_d^{\text{val}}) \leftarrow \text{split}(G_d, p = 0.10)$
7. epochs $\leftarrow 80$ if $d = 0$ else 25
8. Train θ with focal loss on G_d^{tr} , early-stop on val-trader-recall(G_d^{val}), patience 6
9. $\theta_d \leftarrow \theta$; persist checkpoint
10. **end for**

Algorithm: Per-trader engineered feature extraction.

1. **Input:** orders O , trades T for one run; trader v
2. **Output:** $\phi_1, \dots, \phi_6$ for v
3. $O_v \leftarrow$ orders of v in O
4. $B_v \leftarrow$ argmax over 5-min windows of $|O_v \cap \text{window}|$
5. $T_v \leftarrow$ trades involving v during B_v
6. $\phi_1 \leftarrow |B_v|/|O_v|$
7. $p \leftarrow$ buy-fraction in B_v ; $\phi_2 \leftarrow -p \log_2 p - (1 - p) \log_2 (1 - p)$
8. $s_k \leftarrow$ counterparty- k share of T_v ; $\phi_3 \leftarrow \sum_k s_k^2$; $\phi_5 \leftarrow \max_k s_k$
9. $\phi_4 \leftarrow \sigma(q)/\mu(q)$ for order quantities q in O_v
10. $A \leftarrow$ top-quartile-activity traders active during B_v ; $\phi_6 \leftarrow |A \setminus \{v\}|$

5.6 Training Protocol

The training protocol is summarised in Algorithm ?? . Five days of cohort data are processed sequentially. Day 0 is trained from scratch with the full epoch budget; days 1–4 warm-start from the previous day’s checkpoint with a smaller epoch budget to limit catastrophic forgetting. Early stopping is governed by per-family trader recall on an in-day validation split rather than by validation loss, in keeping with the surveillance operating metric.

5.7 Feature Computation

Each of the six engineered features in §5.2 is computed from order- and trade-level events scoped to a per-trader 5-minute burst window. Algorithm ?? sketches the procedure. The burst window is selected as the contiguous 5-minute slice with the highest order count for that trader. Features are computed per run and cached, so repeated experiments incur only one feature-computation pass per cohort regardless of model.

Table 3: Hyperparameters held constant across all comparisons.

Component	Parameter	Value
GraphSAGE	layers	2
	hidden dims	256, 128, 64
	aggregator	max
	dropout	0.3
	loss	focal ($\alpha=0.85, \gamma=2.0$)
	early-stop patience	6 (val trader recall)
Tier-2 GBM	n_estimators	120
	max_depth	3
	learning_rate	0.08
	subsample	0.85
Both	random seed	42

6 Implementation and Reproducibility

6.1 Hardware and Software Stack

All graph network training and inference runs on an NVIDIA RTX 5080 workstation card (sm_120 architecture, 16 GB VRAM). The training pipeline is containerised with Docker; the GraphSAGE trainer uses PyTorch 2.x and PyTorch Geometric. The stacked classifier uses scikit-learn’s **GradientBoostingClassifier**. Per-run graph construction is pure NumPy/pandas and is independent of the deep-learning stack, which keeps the feature pipeline CPU-portable.

6.2 Hyperparameters

Table 3 lists the hyperparameters held constant across all comparisons. The graph network’s loss configuration was selected in an earlier ablation in which a softer focal ($\gamma=1.0, \alpha=0.75$) and longer patience were both empirically worse than the baseline. The boosted classifier’s settings are scikit-learn defaults adjusted for the dataset size; we did not perform a hyperparameter search beyond a small initial grid for the number of estimators.

6.3 Runtime

A single five-day continual training run for the augmented network takes approximately 170 minutes on the workstation card. Feature caching for the OOD cohort completes in under two minutes on CPU. The boosted classifier under leave-one-run-out cross-validation (50 folds, $\approx 10,000$ trader-rows) completes in under 45 seconds on a single CPU core. Inference on the 50-run OOD cohort for either model takes well under a second on GPU. These modest runtimes mean the entire ablation matrix in this paper was produced from disk-cached graphs and features in a few hours

Table 4: Within-generator OOD results, fifty runs.

Metric	v1 (2-feat)	v4 (8-feat)
Edge AUC	0.638	0.984
Clique recall	0.406	0.810
Ring recall	0.480	0.928
Mixed recall	0.408	0.745
Clique purity	0.113	0.774
Ring purity	0.136	0.729
Mixed purity	0.094	0.705

of wall-clock time, supporting frequent iteration.

7 Experiments

7.1 Hyperparameter Ablation

We first verified that the baseline GraphSAGE’s hyperparameters were near-optimal before declaring the model the ceiling. Three single-knob variants were re-trained: v1 (the baseline; focal $\gamma = 2.0, \alpha = 0.85$, patience = 6), v2 ($\gamma = 1.0, \alpha = 0.75$, patience = 12), and v3 ($\gamma = 2.0, \alpha = 0.85$, patience = 12). Trader-level AUCs were 0.612, 0.486, and 0.083 respectively. The patience knob alone collapsed the model. We concluded that hyperparameter tuning had hit a ceiling and that further improvement would require either richer features or richer model structure.

7.2 Within-Generator OOD Result

The within-generator out-of-distribution result is the primary headline of the study. Table 4 compares the baseline network against its feature-augmented re-training on the fifty-run OOD cohort. Edge AUC climbs from 0.638 to 0.984; per-family trader recall climbs from 0.41–0.48 to 0.74–0.93; per-family trader purity climbs from 0.09–0.14 to 0.71–0.77. On average, v4 catches 83% of manipulators at 74% purity, while the baseline caught 43% at 11%—i.e. nine of every ten flags were false positives. The single change was the node feature vector.

7.3 Tier-2 Stacked Classifier

Under leave-one-run-out cross-validation across the same fifty OOD runs, the second-stage classifier consuming the baseline’s score plus the six engineered features achieves pooled trader-level AUC 0.968—a substantial lift from the baseline’s 0.793 trader-level AUC on the same data. At 99% precision the classifier catches 62% of all manipulators in the cohort; at 95% precision, 74%; at the operating point of best F_1 , 76% of manipulators at 94% purity. The lift over the baseline (which caught 11.7% at the threshold typical of

Table 5: Leave-one-feature-out ablation, tier-2 classifier.

Feature dropped	Δ AUC	Rec.@p99
full (7 features)	—	0.619
ϕ_1 burst concentration	−0.051	0.274
ϕ_6 co-active count	−0.007	0.592
ϕ_2 side entropy	−0.004	0.634
ϕ_4 order qty CoV	−0.003	0.608
ϕ_3 counterparty HHI	−0.002	0.620
ϕ_5 top-partner share	+0.000	0.625
x_0 baseline v1 score	+0.000	0.530

its own score distribution) is roughly six-fold in recall at near-identical purity.

We attribute the lift to three converging causes: feature stacking (no single feature exceeds 0.800 standalone), non-linear interaction modelling (a logistic regression on the same seven features reaches AUC ≈ 0.93 , so the boosted trees buy roughly four more AUC points), and the differing error structure of the baseline network versus the trader-marginal features. The network’s false positives tend to be high-activity benign traders; the HHI feature’s false positives tend to be traders with one or two favourite counterparties; the overlap is mostly real manipulators.

7.4 Feature Ablation Inside the Stacked Classifier

A leave-one-feature-out ablation through the stacked classifier reveals an uneven distribution of load (Table 5). Dropping ϕ_1 (burst concentration)—the fifth-ranked feature by standalone AUC—costs −0.051 AUC, the largest single drop, and collapses recall@99%-purity from 0.62 to 0.27. Dropping ϕ_2 (side entropy)—the worst feature standalone—costs only −0.004 AUC. The standalone ranking is therefore a poor proxy for ensemble importance: features that are weak in isolation can be load-bearing in combination.

7.5 Cross-Generator (ABIDES) Result

Evaluating both detectors on the fifteen-run expanded ABIDES cohort yields the comparison in Table 6. The augmented network retains edge AUC 0.842 on this genuinely unseen distribution; the unaugmented baseline collapses to chance (0.518). The +0.324 AUC gap between the two models is consistent across all runs (no fold sees the baseline above 0.55). Per-family v4 mean recall remains high on ABIDES: ring 1.000 ± 0.000 across five runs, mixed 0.983 ± 0.015 , and clique 0.905 ± 0.113 .

Purity, however, does *not* transfer. Both detectors land in the 0.10–0.30 purity range on ABIDES—far

Table 6: Cross-generator (ABIDES expanded, $n = 15$) result.

Metric	v1 ABIDES	v4 ABIDES
Edge AUC	0.518	0.842
Clique recall (mean $\pm$ sd)	0.867 ± 0.16	0.905 ± 0.11
Ring recall	0.864 ± 0.03	1.000 ± 0.00
Mixed recall	0.837 ± 0.07	0.983 ± 0.02
Clique purity (aggregate)	0.096	0.094
Ring purity	0.276	0.293
Mixed purity	0.249	0.269

Table 7: Family-disjoint AUC. Tier-2 is leave-one-family-out CV; v4 is end-to-end retraining with the family filtered from training data.

Held-out family	Tier-2 GBM AUC	v4 GraphSAGE AUC
Ring	0.994	0.480
Clique	0.975	0.533
Mixed	0.906	0.374

below the 0.70–0.77 v4 achieves within-generator. The convergence of both models on similar low purity suggests this is a property of the *cohort* (i.e., a locked threshold tuned in-sample on a smaller cohort happens not to separate ABIDES benigns from manipulators cleanly) rather than a v4-specific weakness. Cross-generator deployment would therefore require per-generator threshold recalibration, an honest finding that we surface explicitly rather than paper over.

7.6 Family-Disjoint Result

The most surprising result of the study comes from the leave-one-family-out evaluation. For the stacked classifier of §5.3 we trained three folds: one in which all clique runs were excluded from training and the model was evaluated on the held-out clique runs, and analogously for ring and mixed. The folds for the end-to-end v4 network were generated identically, by adding a `HOLDOUT_FAMILY` filter to the trainer that excluded all runs of that family from training. Each model was then evaluated on the full fifty-run cohort.

Table 7 contrasts the two architectures. The stacked classifier retains AUC 0.91–0.99 on each held-out family, with the ring fold actually exceeding the baseline leave-one-run-out reference (0.994 vs. 0.968). The end-to-end network, by contrast, collapses to chance or anti-chance on every fold.

A methodological caveat applies to the v4 result. Excluding a family from training reduces each per-day cohort from approximately fifty runs to thirty-three, and the validation-set sampling correspondingly shrinks from five runs to three. With only three validation graphs, the early-stopping signal (mean per-family trader recall on validation) saturates after one

or two epochs, terminating training at eight epochs of overfitting on a tiny in-family sample. A longer training schedule with looser early stopping might recover some performance and remains worth investigating. Nevertheless, the contrast with the stacked classifier on the same task is unambiguous: under the standard training protocol that produced the headline v4 OOD result, family disjointness is fatal to the end-to-end network.

7.7 Per-Run Variance on the ABIDES Cohort

The per-run breakdown of the cross-generator result deserves attention because three of the fifteen ABIDES runs are responsible for most of the spread in clique recall. Across the five ring runs, v4 recall is exactly 1.000 with zero variance; across the five mixed runs, recall is 0.98 ± 0.02 ; across the five clique runs, recall is 0.91 ± 0.11 with one outlier at 0.69. The outlier corresponds to a clique scenario whose front counterparties happened to receive unusually balanced order flow, partially masking the burst-concentration signature. The remaining four clique runs all exceed 0.85. This pattern, where the same metric is bimodal across runs, recurs in the within-generator OOD cohort as well, and is a fair argument for reporting cohort-mean numbers alongside per-run distributions.

7.8 Case Study: A Successful Ring Detection

We pick a representative ring run from the within-generator OOD cohort and trace why v4 succeeds. The run contains ~ 200 traders, six of whom are members of a ring rotating positions among themselves over a single ~ 45 -minute window. The six manipulator traders share three salient properties: (i) extreme burst concentration $\phi_1 > 0.85$ (all six)—versus a benign median of 0.16; (ii) low counterparty HHI $\phi_3 < 0.2$, reflecting the ring’s intentionally diversified counterparty list, versus a benign median near 0.5 (note the manipulator direction is *low*, since genuinely focused traders tend to have a small partner set, the opposite of what the textbook intuition would predict); (iii) high co-active count ϕ_6 because the six rotate trades among themselves and against each other simultaneously. The augmented network’s first SAGE-Conv layer absorbs these node-level signals, and the message-passing step pools each ring member’s neighbourhood (which is the rest of the ring), reinforcing the classification. The same six traders are flagged by the second-stage classifier independently on the trader-marginal features alone.

7.9 What Happens When v4 Fails

In the family-disjoint regime, v4’s early-stopping signal hits a plateau-like maximum within two epochs. This is unsurprising once the val set is reduced to three graphs: with $\sim 1,200$ trader nodes total and a handful of positives, any non-trivial scoring function will achieve perfect trader recall on the val set at some threshold. The patience-6 counter then expires without progress, and training terminates at epoch 8. The remaining 75% of the epoch budget is unused. We believe this is the primary reason the v4 family-disjoint result collapsed, and that a longer schedule or larger val pool may substantially change the picture.

8 Discussion

The pattern of results across these three out-of-distribution regimes suggests a clear architectural distinction. The engineered manipulation-signature features carry signal that is *manipulation-essential rather than family-specific*: tier-2 on those features generalises across families well above the all-family leave-one-run-out reference. Yet the end-to-end network, which consumes the same six features as node attributes, does not generalise across families. The most natural explanation is that the GNN’s message-passing layers learn to recognise family-specific structural neighbourhoods (a ring’s closed cycle is structurally different from a clique’s shared-front arrangement), and when one family is held out the network’s structural prior simply has no correspondence to the held-out pattern.

The bolt-on classifier in contrast sees each trader independently, with no notion of structural neighbourhood at all—it cannot use the neighbourhood signal, but it cannot be misled by it either. The trader-marginal features are family-agnostic by construction (burst concentration, side entropy, etc. describe what manipulation looks like, not what clique manipulation looks like), and the GBM learns combinations that transfer.

This is the most actionable finding of the study. For deployments where the operational distribution may differ structurally from the training distribution—a different exchange, a different ticker universe, a different manipulation taxonomy than was anticipated at training time—a simpler classifier on top of taxonomy-grounded features may be substantially more robust than a more expressive end-to-end network. The latter wins comfortably within-distribution, but its advantage is concentrated in regimes where the training and test distributions share structural priors.

The cross-generator (ABIDES) result occupies an interesting middle ground: the engineered features generalise (v4 holds AUC 0.842 versus the baseline’s

chance) but the operating point does not (purity collapses uniformly). This is consistent with the interpretation that *ranking* is preserved across the cross-generator shift but the absolute manipulator score distribution is shifted enough that a threshold tuned on one cohort flags too many benigns on the other. In production, this argues for per-cohort threshold calibration as part of deployment hygiene rather than for retraining at every shift.

9 Limitations and Future Work

Several caveats accompany the present results. First, both the primary generator and ABIDES are synthetic; no real-data labelled cohort is involved, and although the synthesiser is calibrated to NSE bhavcopy moments, real markets contain heavy-tailed events and microstructure shifts that synthetic data captures only approximately. Second, the ABIDES cohort, although now at $n = 15$, remains modest; expanding to fifty or more runs would tighten the confidence intervals on the cross-generator finding. Third, the v4 family-disjoint result was obtained under the standard training schedule of the main OOD run, and a separate study with a longer schedule and larger validation set is indicated before the architectural conclusion is locked in.

Future work falls into three threads. The most direct is a real-data pilot, in which the detectors are run on labelled real-market data (where labels can be obtained from regulatory enforcement actions); the absence of labelled real manipulation data has been a recurring limitation. Second, the family-disjoint v4 retraining should be redone with a larger validation pool (e.g. 20% per day) and longer patience (e.g. 20 epochs), to determine whether v4’s collapse is intrinsic or an early-stopping artefact. Third, end-to-end models may benefit from family-agnostic structural priors—e.g. randomly permuting node features between families during training—to force the message-passing layers to be less family-specific.

10 Threats to Validity

We highlight four threats to the generalisability of the conclusions above. First, the locked decision threshold for both detectors is tuned in-sample on the evaluation cohort, which inflates absolute purity numbers; the cross-MODEL comparison is fair (both detectors suffer the same way), but the absolute operating point is mildly optimistic. Second, the cross-generator cohort, while drawn from a genuinely different simulator, is still synthetic. Real-market manipulators may exhibit microstructure patterns absent from both synthesisers.

Third, the family-disjoint v4 retraining used the training schedule tuned for the original five-day cohort, and the schedule’s interaction with a reduced cohort and reduced validation pool is responsible for at least some of the observed collapse. A follow-up retraining with a relaxed early-stopping criterion is indicated. Fourth, the engineered features were designed by hand based on a taxonomy of manipulation patterns, which is itself a form of inductive bias; features tailored to a different taxonomy would yield a different decision surface.

11 Related Threads Not Pursued

For transparency we list two directions we did not pursue here but which would naturally extend the work. The first is a benign-only cohort, in which every run contains no manipulators; running either detector against such a cohort would yield a clean false-positive rate estimate at any chosen threshold and is the proper way to report the “cost” side of a flagging policy. The second is a study of detector behaviour against an adaptive adversary, in which the manipulator agents themselves are trained or hand-tuned to evade the detector. We note both directions but defer them to subsequent work.

12 Conclusion

We have presented a multi-cohort study of a two-layer GraphSAGE detector augmented with six manipulation-signature features and compared it against a stacked second-stage classifier consuming the same features. The augmented network achieves edge AUC 0.984 and recall 0.83 at purity 0.74 on a fifty-run within-generator OOD cohort, lifting from a two-feature baseline of 0.638/0.43/0.11. The stacked classifier achieves pooled AUC 0.968 with 62% recall at 99% purity. Both detectors are then evaluated on a fifteen-run cross-generator cohort drawn from a different agent-based simulator: the augmented network retains AUC 0.842 versus the baseline’s chance, demonstrating that the engineered features carry signal across generators. A leave-one-family-out evaluation reveals an unexpected contrast, with the stacked classifier retaining AUC 0.91–0.99 on each held-out family while the end-to-end network collapses to chance. The simpler, trader-marginal classifier therefore generalises across manipulation styles better than the more expressive end-to-end network. We argue that for surveillance settings where the operational distribution may differ structurally from the training distribution, the bolt-on architecture deserves serious consideration as a deployment choice on robustness

grounds, even when the end-to-end model wins within-distribution comparisons.

References

- [1] S. Khanna and A. Aggarwal, “A survey of market manipulation detection: trends and open problems,” *Journal of Financial Surveillance*, vol. 8, no. 3, pp. 145–168, 2021.
- [2] Y. Cao, Y. Li, S. Coleman, A. Belatreche, and T. McGinnity, “Particle-swarm-optimised support-vector machine for detecting market manipulation,” in *Proc. IEEE Symp. on Computational Intelligence for Financial Engineering*, 2014.
- [3] T. Sun, K. Liu, X. Chen, and H. Ye, “Graph neural networks for financial-fraud detection: a survey,” *ACM Computing Surveys*, 2022.
- [4] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [5] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in *Int. Conf. on Learning Representations (ICLR)*, 2018.
- [6] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *Int. Conf. on Learning Representations (ICLR)*, 2017.
- [7] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, “Focal loss for dense object detection,” in *Proc. IEEE Int. Conf. on Computer Vision (ICCV)*, 2017.
- [8] J. H. Friedman, “Greedy function approximation: a gradient boosting machine,” *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.
- [9] B. Recht, R. Roelofs, L. Schmidt, and V. Shankar, “Do ImageNet classifiers generalize to ImageNet?” in *Proc. Int. Conf. on Machine Learning (ICML)*, 2019.
- [10] P. W. Koh, S. Sagawa, H. Marklund, S. M. Xie, M. Zhang, A. Balsubramani, W. Hu, M. Yasunaga, R. L. Phillips, I. Gao, T. Lee, E. David, I. Stavness, W. Guo, B. A. Earnshaw, I. S. Haque, S. Beery, J. Leskovec, A. Kundaje, E. Pierson, S. Wang, D. Koller, S. Levine, and P. Liang, “WILDS: A benchmark of in-the-wild distribution shifts,” in *Proc. Int. Conf. on Machine Learning (ICML)*, 2021.

- [11] D. Byrd, M. Hybinette, and T. H. Balch, “ABIDES: Towards high-fidelity multi-agent market simulation,” in *Proc. ACM SIGSIM Conf. on Principles of Advanced Discrete Simulation*, 2019.
- [12] R. Cont, “Empirical properties of asset returns: stylized facts and statistical implications,” *Quantitative Finance*, vol. 1, no. 2, pp. 223–236, 2001.
- [13] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. ACM SIGKDD Conf. on Knowledge Discovery and Data Mining*, 2016.
- [14] J. Wang, R. Wen, C. Wu, Y. Huang, and J. Xiong, “FdGars: Fraudster detection via graph convolutional networks in online app reviews,” in *Companion Proc. The World Wide Web Conference (WWW)*, 2019.
- [15] Y. Liu, X. Ao, Z. Qin, J. Chi, J. Feng, H. Yang, and Q. He, “Pick and choose: A GNN-based imbalanced learning approach for fraud detection,” in *Proc. The World Wide Web Conference (WWW)*, 2021.
- [16] J. Leskovec and R. Sosič, “SNAP: A general-purpose network analysis and graph-mining library,” *ACM Trans. Intell. Syst. Technol.*, vol. 8, no. 1, pp. 1–20, 2016.
- [17] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, “How powerful are graph neural networks?” in *Int. Conf. on Learning Representations (ICLR)*, 2019.
- [18] W. Cong, H. K. Suarez, M. Mahdavi, L. Akoglu, and M. Mahdavi, “AMLSim: a synthetic data generator for anti-money-laundering research,” *IEEE BigData Workshop on Financial Fraud*, 2020.
- [19] Y. Dou, Z. Liu, L. Sun, Y. Deng, H. Peng, and P. S. Yu, “Enhancing graph neural network-based fraud detectors against camouflaged fraudsters,” in *Proc. ACM Int. Conf. on Information and Knowledge Management (CIKM)*, 2020.
- [20] H. Nguyen, P. Phung, and Y. Wang, “Towards out-of-distribution generalisation in graph neural networks: a benchmark,” in *Proc. Int. Conf. on Learning Representations (ICLR) Workshop on ML for Finance*, 2023.